

Federated Learning Model Optimization over Next-Gen Wireless Networks: A Comprehensive Survey

Reza Jahani
Dept. of ECE
North Carolina State University
Raleigh, NC, USA
rjahani@ncsu.edu

Zizhong Wang
Dept. of CS
North Carolina State University
Raleigh, NC, USA
zwang269@ncsu.edu

Zayaan Husain Siddiqui
Dept. of CS
North Carolina State University
Raleigh, NC, USA
zsiddiq@ncsu.edu

Abstract—Federated Learning (FL) has emerged as a key paradigm for enabling distributed intelligence in Next-Generation (NextG) wireless networks by allowing multiple clients to collaboratively train models without sharing raw data. While FL preserves privacy and reduces data transfer, its deployment over resource-limited wireless environments introduces major optimization challenges concerning latency, communication efficiency, and security. Recent studies have focused on developing frameworks that jointly optimize communication and computation, adopt adaptive aggregation mechanisms, and integrate privacy-preserving techniques to address these issues. This survey provides a systematic overview of advancements in model optimization for FL over NextG wireless networks, focusing on three main directions: latency minimization, communication-efficient FL, and secure FL frameworks. After analyzing recent methodologies and the trade-offs between them, this survey highlights key insights and open challenges for achieving robust and secure FL systems in future 6G networks.

Index Terms—Federated Learning, Next-Gen Wireless Networks, Latency Minimization, Communication efficient, Security.

I. INTRODUCTION

A. Background on FL in Wireless Networks

The rapid evolution of wireless communication technologies toward Next-Generation (NextG) systems has given rise to highly distributed environments with huge load of data, where billions of edge devices generate and process data continuously. Traditional centralized machine learning approaches rely on aggregating raw data at a central server, introducing significant concerns regarding privacy, latency, and bandwidth consumption. Federated Learning (FL) was introduced to resolve these limitations by enabling devices to collaboratively train a shared global model with the help of a central server while keeping local data private [1].

In a common FL framework, each participating device performs local training on its dataset and uploads model updates to a central server. The server then computes a global model, commonly using the Federated Averaging (FedAvg) algorithm [2], and broadcasts the updated parameters back to the clients. This iterative process enables distributed knowledge sharing without direct data transmission, preserving user

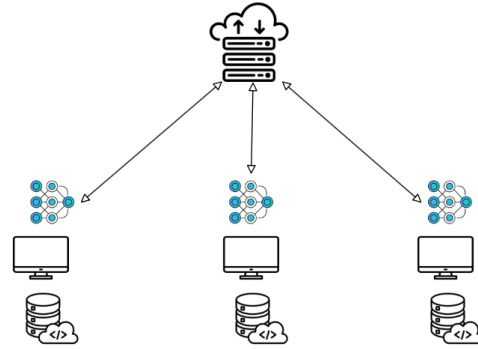


Fig. 1. Federated Learning and collaborative training workflow

privacy and reducing communication overhead. Fig. 1 shows a high-level illustration of FL systems workflow, operating based on collaborative training.

Deploying FL over wireless networks extends this paradigm to practical communication environments, where clients correspond to mobile devices, IoT nodes, sensors, or autonomous vehicles connected through unreliable and heterogeneous communication links. Integrating FL into such systems must be aligned with the design principles and performance requirements of the next generation of wireless networks, which are ubiquitous intelligence, ultra-low latency, and massive device connectivity [3]. Consequently, FL is now considered as a suitable platform for deploying distributed intelligence in NextG wireless systems, empowering emerging applications such as vehicular edge learning, digital twins, and network automation [4].

Despite the benefits FL offers, considerable challenges are introduced to the system. The stochasticity of wireless channels, users mobility, and different computation capabilities can all degrade training efficiency and convergence. These practical issues introduce communication bottlenecks and privacy vulnerabilities which require robust algorithmic design. As a result, model optimization in FL over NextG networks

has become a vital research direction, aiming to enhance scalability, efficiency, and security while meeting the stringent quality-of-service requirements of modern wireless systems.

B. FL performance requirement in Next-Gen networks

NextG wireless networks, 6G and beyond, are expected to support a broad spectrum of intelligent applications such as autonomous driving, digital twins, remote surgery, immersive VR/AR, and IoT. These applications require extremely stringent quality-of-service (QoS) parameters, including ultra-low latency, high reliability, energy efficiency, and strong security guarantees [3]. Achieving such performance in decentralized environments with limited bandwidth demands a tight integration between machine learning models and communication network optimization.

As FL is deployed in dynamic wireless networks, system-level optimization becomes vital to ensure required performance regarding model convergence, communication efficiency, and data confidentiality. These requirements motivate three fundamental research directions that define the focus of this survey [5].

1) **Latency Minimization:** Latency in FL corresponds to the total delay per one round of FL process, which directly influences the convergence time. In these wireless settings, latency is dependent on both uplink and downlink transmission delays and local computation delay raised from heterogeneous devices with different computation capability. Real-time applications such as vehicular coordination require the end-to-end latency to be on the order of milliseconds [4]. Researchers have explored computation offloading, client selection, and joint communication-computation scheduling to minimize per-round and global training latency. Optimizing latency also involves considering and resolving stochastic wireless channels, limited bandwidth, and user mobility [6].

2) **Communication-Efficient Federated Learning:** Communication overhead is one of the primary bottlenecks in large-scale FL deployments. In every training round, a large volume of model parameters must be exchanged between edge devices and the server, using limited wireless bandwidth, thus increasing convergence time. This challenge becomes particularly severe in dense 6G networks where thousands of edge devices may participate. To address this, recent studies propose compression, quantization, and sparsification techniques that reduce message size without impacting model accuracy [7]. Therefore, optimization of communication aspect of FL plays a key role in scaling FL for real-world wireless deployments.

3) **Security and Privacy-Preserving Federated Learning:** Despite FL's privacy-preserving advantages, it remains vulnerable to several attacks such as model inversion, membership inference, and poisoning, which can expose sensitive information or corrupt the global model. Therefore, ensuring secure and trustworthy learning is a crucial design goal in NextG environments that handle confidential data from healthcare, or industrial systems. Researchers have developed privacy-preserving frameworks employing differential privacy, secure

multi-party computation, and homomorphic encryption to protect the information during model exchange [5]. Moreover, robust aggregation and blockchain-based auditing based methodologies have been introduced to prevent model manipulation and ensure accountability among distributed participants [8]. Achieving reliable privacy and security, however, introduces trade-offs with communication and latency.

In summary, latency, communication, and security form the three core directions determining the efficiency and practicality of FL in NextG wireless networks. Understanding the optimization methods proposed for each of these directions is therefore critical for designing scalable and robust intelligent networks.

II. ADVANCEMENTS IN LATENCY MINIMIZATION

Latency minimization plays a vital role in FL over wireless networks, particularly in next generation of wireless networks with a broad range of applications with ultra-low latency requirements and different scenarios involving mobile and heterogeneous clients. Efficient latency control methodologies ensure a convergence rate based on practical constraints while maintaining the required system accuracy. This section reviews three state-of-the-art frameworks that address the latency optimization challenge in different network settings: (1) Client Selection for FL with Heterogeneous Resources in Mobile Edge Networks, (2) Latency Optimization for Wireless FL in Multihop Networks, and (3) WHALE-FL (Wireless and Heterogeneity-Aware Latency Efficient FL). Each framework provides a different perspective and optimization approach to reduce FL delay under particular system constraints.

A. Client Selection for Federated Learning with Heterogeneous Resources in Mobile Edge Networks

FedCS [9], introduces a latency-efficient FL protocol characterized for mobile edge computing (MEC) systems with heterogeneous clients with different dataset sizes, computational capabilities, and channel conditions. The key goal is to maximize the number of participating clients in aggregation step by choosing the clients capable of completing training and transmission within a round deadline T_{round} . This perspective maintains the overall system performance while considering the latency constraint.

1) **Framework and Motivation:** Traditional FL randomly selects clients in each round, ignoring differences in users condition such as computational capabilities, dataset sizes, and channel conditions. This leads to straggling clients that delay aggregation. FedCS mitigates this by introducing an MEC-assisted client selection framework. Fig. 2 illustrates the FedCS protocol workflow.

2) **FedCS Protocol Overview:** The FedCS protocol modifies the standard FL process by replacing the conventional methodology of random client selection used in traditional FL with two new steps: *Resource Request* and *Client Selection*. The steps are summarized as follows:

1) **Initialization:** The server initializes a global model w_0 .

- 2) **Resource Request:** A random subset of clients $\mathcal{K}' \subseteq \mathcal{K}$ (with $|\mathcal{K}'| = C \times |\mathcal{K}|$) report their resource information, including computing power, and data size.
- 3) **Client Selection:** The MEC operator selects a subset $\mathcal{S} \subseteq \mathcal{K}'$ that can complete the training process within T_{round} , maximizing the number of accepted clients $|\mathcal{S}|$.
- 4) **Distribution:** The BS multicasts the global model to selected clients.
- 5) **Scheduled Update and Upload:** Selected clients in $\mathcal{S}$ perform local updates and *sequentially* upload model parameters using the allocated resource blocks (RBs).
- 6) **Aggregation:** The server aggregates received models using standard FedAvg [2].

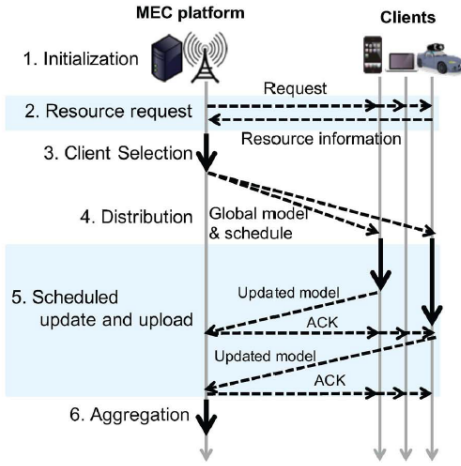


Fig. 2. FedCS Protocol Workflow

FedCS aims to maximize the number of participating devices by choosing the clients who can complete steps 4 and 5 within a deadline. In the client selection step, a subset of users $\mathcal{S}$ are chosen where the users were in $\mathcal{K}'$.

Each client k reports its resource information, including local update time t_k^{UD} and upload time t_k^{UL} . The MEC operator estimates whether a client can finish within T_{round} . The total elapsed time for the i -th client completing the *Scheduled Update and Upload* step is defined recursively as:

$$\Theta_i = T_i^{UD} + T_i^{UL} = \sum_{j=1}^i (\max\{0, t_{k_j}^{UD} - \Theta_{j-1}\} + t_{k_j}^{UL}), \quad (1)$$

Where $t_{k_i}^{UD}$ and $t_{k_i}^{UL}$ denote the local training time and the model upload time for client k_i , respectively. This formulation allows the clients to perform the local update simultaneously. However, this framework distributes the RBs for communication in a way that the uploading step is done sequentially for each user.

3) **Optimization Formulation:** The objective is to select as many clients as possible while ensuring that the total round

duration does not exceed T_{round} :

$$\max_{\mathcal{S}} |\mathcal{S}|, \quad (2)$$

$$\text{s.t. } T_{\text{round}} \geq T_{\text{CS}} + T_{\mathcal{S}}^D + \Theta_{|\mathcal{S}|} + T_{\text{agg}}, \quad (3)$$

where T_{CS} , $T_{\mathcal{S}}^D$, and T_{agg} denote the durations for client selection, model distribution, and aggregation, respectively. This combinatorial problem is NP-hard since both client subset $\mathcal{S}$ and upload order affect $\Theta_{|\mathcal{S}|}$.

4) **Greedy Client Selection Algorithm:** FedCS applies a greedy heuristic inspired by knapsack optimization [10]. Clients are iteratively added to $\mathcal{S}$ by selecting:

$$x = \arg \min_{k \in \mathcal{K}'} (T_{\mathcal{S}} + t_k^L + \max\{0, t_k^{UD} - \Theta\}), \quad (4)$$

A client is added if the total estimated time remains below T_{round} . The algorithm complexity is $\mathcal{O}(|\mathcal{K}'||\mathcal{S}|)$, significantly less than naive brute force $\mathcal{O}(2^{|\mathcal{K}'|})$ [9].

5) **Results and Findings:** Simulations were conducted on a realistic LTE-based MEC environment with $K = 1000$ clients using CIFAR-10 and Fashion-MNIST datasets. FedCS achieved substantially faster convergence than baseline FedLim (FedAvg with deadlines), reducing time-to-accuracy (ToA) by 76.5 minutes on CIFAR-10 and 33 minutes on Fashion-MNIST while achieving comparable final accuracy. It also demonstrated robustness under non-IID data distributions. Illustrated in Fig. 3, FedCS achieves the desired accuracy within a shorter time compared to FedLim.

Method	CIFAR-10		
	ToA@0.5	ToA@0.75	Accuracy
FedLim ($T_{\text{round}} = 3 \text{ min}$)	38.1	209.2	0.77
FedCS			
$T_{\text{round}} = 3 \text{ min}$ ($r = 0\%$)	25.8	132.7	0.79
$T_{\text{round}} = 3 \text{ min}$ ($r = 10\%$)	27.9	138.1	0.78
$T_{\text{round}} = 3 \text{ min}$ ($r = 20\%$)	31.1	178.3	0.78
Method	Fashion-MNIST		
	ToA@0.5	ToA@0.85	Accuracy
FedLim ($T_{\text{round}} = 3 \text{ min}$)	10.4	66.8	0.90
FedCS			
$T_{\text{round}} = 3 \text{ min}$ ($r = 0\%$)	10.6	33.5	0.91
$T_{\text{round}} = 3 \text{ min}$ ($r = 10\%$)	11.3	32.1	0.92
$T_{\text{round}} = 3 \text{ min}$ ($r = 20\%$)	12.7	37.0	0.91

Fig. 3. Results obtained for CIFAR-10 and F-MNIST with FedCS

B. Latency Optimization for Wireless Federated Learning in Multihop Networks

Authors in [11] propose a comprehensive optimization framework for minimizing latency in multi-hop wireless FL systems through joint leaf-relay optimization and adaptive routing. The system consists of M leaf nodes and N relay nodes distributed over R routes from devices to the server. Each leaf node performs local training and uploads model parameters via relay nodes participating in forwarding and aggregation, where each relay node performs local training in addition to passing the model forward towards the server.

1) *Framework and Personalized Model*: The authors develop a **Personalized and Adaptive Aggregation-aware Federated Learning (PAFL)** methodology to resolve data heterogeneity. The local update rule for client u at global round k and local iteration t is:

$$\mathbf{w}_{u,k}^{t+1} = \mathbf{w}_{u,k}^t - \eta [\mathbf{g}_{u,k}^t + \lambda(\mathbf{w}_{u,k}^t - \mathbf{w}_k)], \quad (5)$$

where $\mathbf{w}_{u,k}^t$ is the local model, $\mathbf{w}_k$ is the global model, η is the learning rate, and λ controls the balance between personalization and global regularization. The global aggregation is done based on weighted averaging:

$$\mathbf{w}_{k+1} = \frac{\sum_{u=1}^U \alpha_u \mathbf{w}_{u,k}^T}{\sum_{u=1}^U \alpha_u}, \quad (6)$$

where α_u corresponds to the importance of client u , further controlling the effect of each client on the global model.

2) *System Latency Model*: Authors involve the number of required iterations and dataset size in addition to CPU characteristics of each leaf node m and relay node n in the computation and communication latency:

$$T_m^k = \frac{L_m^k C_m^k D_m^k}{f_m^k} + \frac{s}{R_m^k}, \quad (7)$$

$$T_n^k = \frac{L_n^k C_n^k D_n^k}{f_n^k} + \frac{s}{R_n^k} + \frac{sn'}{R_n^k}, \quad (8)$$

where L_m^k is the number of local iterations, C_m^k is CPU cycles per sample, D_m^k is data size, f_m^k is CPU frequency, R_m^k is uplink rate, and n' represents the number of successor nodes per relay. The total latency for route r and global round k is:

$$T_{\text{total}}^k = \max_{r \in \mathcal{R}} \left(T_m^k + \sum_{n=1}^N T_n^k \right). \quad (9)$$

3) *Optimization Problem*: The latency minimization problem is formulated as:

$$\min_{\{p, f, \delta\}} T_{\text{FL}}^{\text{total}} = \sum_{k=1}^K T_{\text{total}}^k, \quad (10)$$

$$\text{s.t. } E_m^k \leq E_m^{\text{max}}, E_n^k \leq E_n^{\text{max}}, \quad (11)$$

$$0 \leq f_m^k \leq F_m, 0 \leq f_n^k \leq F_n, \quad (12)$$

$$\delta_{r,n}^k \in \{0, 1\}. \quad (13)$$

Here, $\delta_{r,n}^k$ is a binary routing indicator which determines the participation of the relay node n in round k in route r , and E_m^k, E_n^k denote energy constraints relative to the discussed parameters of leaf and relay nodes.

The problem is categorized as a non-convex problem. To solve it, the authors apply Block Coordinate Descent (BCD) and Successive Convex Approximation (SCA). The solution iteratively optimizes routing, leaf-node, and relay-node variables until convergence with employing alternation minimization [12] approach, i.e., forming a convex optimization problem by fixing other parameter and then applying a convex optimization method to find the optimum value for each parameter. This algorithm is performed iteratively over all the parameters until convergence.

4) *Experimental Results*: Simulation results illustrate that the proposed algorithm reduces latency by up to 69.37% compared to the baseline. Integration of energy harvesting into relays further enhances sustainability, achieving additional reductions as the number of nodes increases. The framework demonstrates significant convergence and latency gains under non-IID data across MNIST and CIFAR-10 tasks.

C. WHALE-FL: Wireless and Heterogeneity-Aware Latency Efficient Federated Learning

WHALE-FL [13] addresses latency in FL over mobile devices with highly dynamic computing and communication conditions by adaptively assigning sub-networks to the clients. Main idea behind this methodology lies in optimizing the training and transmission time for each user by controlling the complexity of users mode based on personalized conditions, such as computation capability and channel condition. Unlike traditional fixed subnetwork assignment methods, authors introduce an adaptive subnetwork scheduling framework that jointly considers system heterogeneity and FL training dynamics. This adaptive framework handles the trade-off between the contribution of each user and their computation/communication delay.

1) *Adaptive Subnetwork Scheduling Framework*: WHALE-FL allows each device to train a subnetwork extracted from the global model based on its instantaneous resources and learning contribution. The key innovation lies in defining an adaptive subnetwork selection utility function:

$$\text{Util}(i, r) = \underbrace{|B_i| \sqrt{\frac{1}{|B_i|} \sum_{k \in B_i} \sum_{d \in \mathcal{D}} (FI_{i,r-d}(k))^2 / |\mathcal{D}|}}_{\text{Training Efficiency}} \times \underbrace{\left(\frac{T}{T_i^{\text{tr}} + T_i^{\text{co}}} \right)^\beta}_{\text{System Efficiency}}, \quad (14)$$

where $FI_{i,r}$ is the local Fisher Information (capturing training dynamics), T_i^{tr} and T_i^{co} are transmission and computation delays, and β tunes the trade-off between system and training efficiency. This methodology allows the FL system to consider the effectiveness of clients dynamically based on the *Training Efficiency* term in the utility function and adaptively assign them sub-networks to jointly optimize the latency and system accuracy.

Each device normalizes $\text{Util}(i, r)$ to select an appropriate subnetwork size level:

$$\hat{W}(i, r) = \begin{cases} W^1, & U_n(i, r) \geq \frac{P-1}{P}, \\ \vdots \\ W^P, & U_n(i, r) < \frac{1}{P}, \end{cases} \quad (15)$$

where W^p represents the p -th subnetwork extracted from the global model with shrinkage ratio $s^{2(p-1)}$.

2) *Optimization and Implementation*: WHALE-FL operates fully distributed and independent. Each client computes $\text{Util}(i, r)$ locally and autonomously determines its subnetwork size. The FL server aggregates heterogeneous subnetworks as:

$$W_g^{r+1} = W_P^{r+1} \cup \bigcup_{p=2}^P (W_{p-1}^{r+1} \setminus W_p^{r+1}), \quad (16)$$

ensuring compatibility of heterogeneous models.

Task	CV		NLP	HAR
	CNN@MNIST	Resnet@CIFAR10	Transformer@Wiktext2	CNN@HAR
Target Accuracy	85%	70%	37%	88%
Method	Speedup			
WHALE vs HeteroFL	1.74x	1.25x	1.21x	1.06x
WHALERolex vs FedRolex	1.75x	1.32x	1.24x	1.10x
WHALEDropout vs FedDropout	1.70x	1.24x	1.20x	1.05x

Fig. 4. Experimental results for WHALE-FL

3) *Experimental Results*: Fig. 4 shows the comparison between multiple state-of-the-art methods integrated with WHALE-FL methodology and the baseline. Experimental results suggests that incorporating WHALE-FL in FL frameworks enables the system to reduce the latency while maintaining the target accuracy. Experiments over 20 heterogeneous devices (MacBook, Jetson, Raspberry Pi) under Wi-Fi/LTE/Bluetooth environments show that WHALE-FL accelerates FL convergence by up to $1.9\times$ compared with FedAvg and $1.74\times$ compared with HeteroFL, without accuracy degradation. By dynamically adapting subnetwork sizes based on communication and learning dynamics, WHALE-FL achieves superior latency efficiency while maintaining robustness to heterogeneity and non-IID data.

III. ADVANCEMENTS IN COMMUNICATION-EFFICIENT FL

Communication efficiency plays a crucial role in the practical deployment of FL over wireless and resource-constrained environments. Each communication round involves the exchange of model parameters between the central server and edge devices, which can lead to excessive bandwidth consumption and slow convergence, especially in dense NextG wireless systems. To mitigate these issues, recent studies propose strategies such as gradient compression, quantization, asynchronous aggregation, and hierarchical communication scheduling that aim to reduce the amount and frequency of transmitted data without impacting model performance. This section reviews five state-of-the-art frameworks that address communication bottlenecks through novel quantization and aggregation mechanisms: (1) FedPAQ—Periodic Averaging and Quantization, (2) Adaptive Gradient Quantization (AGQ) for Heterogeneous Edge Devices, (3) DAdaQuant—Doubly Adaptive Quantization, (4) FedAT—Asynchronous Tiers for Communication Efficiency, and (5) CHFL—Compressed Hierarchical Federated Learning for Imbalanced Edge Networks.

A. FedPAQ: Periodic Averaging and Quantization

FedPAQ [14] introduces a communication-efficient FL framework that integrates **periodic model averaging** with **gradient quantization** to reduce both the frequency and size of updates exchanged between clients and the server. The main objective is to minimize communication cost while preserving convergence comparable to standard FedAvg.

1) *Framework and Motivation*: Traditional FL protocols perform global aggregation after every local iteration, which incurs frequent communication overhead. FedPAQ decouples computation from communication by enabling each client to perform multiple local updates before transmitting model

parameters. This *periodic averaging* substantially reduces the number of communication rounds. Additionally, before transmission, clients quantize their gradient updates, further minimizing communication payload.

2) *Quantized Update Mechanism*: Each client i performs local SGD for τ steps and sends quantized updates to the server:

$$Q(\mathbf{g}_i) = \text{sign}(\mathbf{g}_i) \cdot \|\mathbf{g}_i\|_p \cdot \xi, \quad (17)$$

where $\mathbf{g}_i$ denotes the local gradient, $\|\cdot\|_p$ represents p -norm scaling, and ξ is the quantization resolution factor. The server aggregates updates every τ steps as:

$$\mathbf{w}_{t+\tau} = \mathbf{w}_t - \eta \sum_{i=1}^N Q(\mathbf{g}_i), \quad (18)$$

where η is the global learning rate.

3) *Optimization and Results*: The trade-off between local update frequency τ and quantization resolution ξ defines overall communication efficiency. Larger τ values reduce communication frequency but may slow convergence, whereas smaller quantization resolution improves bandwidth efficiency but risks gradient distortion. FedPAQ optimizes both to minimize communication cost while maintaining convergence.

Experiments on CIFAR-10 and FEMNIST datasets show that FedPAQ achieves up to **40 $\times$ reduction** in communication volume with less than **1% accuracy loss**.

B. Adaptive Gradient Quantization (AGQ) for Heterogeneous Edge Devices

AGQ [15] proposes an adaptive quantization mechanism that dynamically adjusts bit precision based on each client's wireless link quality and gradient variance. In heterogeneous edge networks, devices vary widely in their computation and communication capabilities, making static quantization inefficient.

1) *Framework Overview*: Each client selects a quantization level depending on its channel quality and gradient dynamics. Devices with high signal-to-noise ratio (SNR) and low gradient variance adopt fewer bits to save bandwidth, while those with noisy channels use higher bit precision for stability.

2) *Adaptive Quantization Strategy*: The bit width b_i for client i is computed as:

$$b_i = b_{\min} + \alpha \cdot \log(1 + \text{SNR}_i), \quad (19)$$

where α is a tunable parameter. Quantization of gradients is defined as:

$$Q_i(\mathbf{g}_i) = \text{clip}\left(\frac{\mathbf{g}_i}{\|\mathbf{g}_i\|_\infty}, -1, 1\right) \cdot 2^{b_i}. \quad (20)$$

3) *Optimization Problem*: The communication optimization is formulated as:

$$\min_{\{b_i\}} \sum_{i=1}^N \frac{|\mathbf{g}_i|}{b_i}, \quad (21)$$

$$\text{s.t. } \mathcal{L}(\mathbf{w}) - \mathcal{L}^* \leq \epsilon, \quad (22)$$

where $\mathcal{L}(\mathbf{w})$ and $\mathcal{L}^*$ respectively represent the global loss evaluated at the aggregated model and optimum value of objective and ϵ bounds the permissible accuracy loss.

4) *Experimental Results*: Simulations over diverse edge setups demonstrate a **60% reduction in communication overhead** compared to full-precision FedAvg, with negligible accuracy loss. AGQ ensures fair participation across clients, mitigating dominance by high-capacity nodes.

C. DAdaQuant: Doubly-Adaptive Quantization for Federated Learning

DAdaQuant [16] presents a **doubly adaptive quantization** framework that adapts precision across both clients and training time. Unlike fixed-bit schemes, DAdaQuant dynamically tunes quantization resolution based on learning rate and accumulated error.

1) *Methodology*: DAdaQuant incorporates two adaptive mechanisms:

- 1) **Learning-rate adaptive quantization**: The quantization precision b_t decays with the learning rate, enabling higher compression in later rounds.
- 2) **Error-feedback compensation**: Clients store quantization residuals to correct future updates, maintaining convergence stability.

The quantized gradient is expressed as:

$$Q(\mathbf{g}_t) = \text{clip}\left(\frac{\mathbf{g}_t}{\alpha_t}, -1, 1\right) \cdot 2^{b_t}, \quad (23)$$

where α_t is a dynamic scaling factor updated as:

$$\alpha_{t+1} = \gamma\alpha_t + (1 - \gamma)\|\mathbf{g}_t\|, \quad b_{t+1} = b_t - \delta \cdot LR_t. \quad (24)$$

2) *Performance Analysis*: Experiments on CIFAR-10 and ImageNet show up to **95% reduction in uplink communication cost** with accuracy comparable to uncompressed FL. The adaptive quantization prevents gradient staleness under non-IID conditions.

D. FedAT: Asynchronous-Tiered Federated Learning System

FedAT [17] introduces a high-performance FL architecture that leverages **asynchronous tiered aggregation** to address communication inefficiencies arising from client heterogeneity and network delays. Unlike synchronous schemes where all clients must wait for slow devices, FedAT divides the FL system into multiple asynchronous tiers to achieve faster global updates.

1) *Framework and Architecture*: FedAT operates through a three-tier architecture consisting of:

- **Device Tier**: Edge devices perform local updates asynchronously.
- **Edge Tier**: Edge servers aggregate local updates from connected devices and perform partial model averaging.
- **Cloud Tier**: The cloud server periodically synchronizes global models across edge servers.

Each tier operates asynchronously, enabling faster nodes to contribute more frequently without being stalled by slower devices.

2) *Optimization Design*: The system employs a staleness-aware weighting mechanism. Updates from slower nodes are discounted by a temporal decay factor:

$$\mathbf{w}_{t+1} = (1 - \mu)\mathbf{w}_t + \mu \sum_i \omega_i \mathbf{w}_i^t, \quad (25)$$

where $\omega_i = e^{-\lambda \Delta t_i}$ adjusts the weight of each client's update based on its staleness Δt_i and decay rate λ .

3) *Experimental Evaluation*: FedAT achieves up to **3.2× faster convergence** and **45% less communication cost** compared to FedAvg in heterogeneous wireless settings. It maintains stability even with 30% of devices operating on delayed updates.

This work demonstrates that asynchronous-tiered FL minimizes idle time and reduces communication synchronization overhead, providing an efficient solution for large-scale mobile networks.

E. CHFL: Compressed Hierarchical Federated Learning for Edge-Level Imbalanced Wireless Networks

CHFL [18] presents a **compressed hierarchical federated learning** framework that integrates local model compression with multi-level aggregation to enhance communication scalability in edge-intense NextG networks.

1) *System Architecture*: The system features three aggregation layers:

- 1) **Client Layer**: Devices locally train and compress model updates.
- 2) **Edge Layer**: Edge servers aggregate client updates and apply further compression before transmission.
- 3) **Cloud Layer**: The global server performs final model averaging.

This structure reduces total data transmitted over long-distance links.

2) *Compression and Aggregation*: Each client applies a top- k sparsification and quantization operator:

$$Q_k(\mathbf{g}_i) = \text{sign}(\mathbf{g}_i^{(k)}) \cdot \|\mathbf{g}_i^{(k)}\|_2, \quad (26)$$

where $\mathbf{g}_i^{(k)}$ retains only the top- k gradients by magnitude. Edge servers compress aggregated results using a secondary quantization layer before uploading to the cloud.

3) *Results and Findings*: Simulations under imbalanced data conditions show CHFL achieves **80% bandwidth reduction** and **1.5× faster convergence** compared to conventional hierarchical FL. The hierarchical compression strategy effectively mitigates uplink congestion while maintaining accuracy.

Summary. Communication-efficient FL solutions employ quantization, adaptive scheduling, and hierarchical aggregation to reduce bandwidth usage while preserving model performance. Future NextG networks will likely adopt hybrid frameworks combining these strategies to enable scalable, real-time, and energy-efficient federated intelligence across massive device populations.

IV. ADVANCEMENTS IN SECURE FL

Although FL enables decentralized training while preserving data locality, it remains exposed to various security and privacy threats such as poisoning, backdoor, and inference attacks. In NextG wireless environments, which are characterized by large-scale deployments of heterogeneous and potentially distrusted edge devices, such vulnerabilities introduce significant threats to both the integrity of the learned models and the reliability of the overall system.

NextG applications, including autonomous vehicles, industrial IoT, and healthcare networks, demand secure and efficient learning under bandwidth, latency, and resource constraints. Therefore, modern FL frameworks must integrate security mechanisms that are adaptive, lightweight, and optimization-aware, ensuring robustness without compromising convergence or communication efficiency.

This section reviews representative frameworks that enhance FL security and privacy in resource-constrained and adversarial settings. The discussed approaches span four main directions: (1) robust aggregation and anti-poisoning defenses, (2) model pruning for efficient and secure federated learning, and (3) differential privacy for Federated Learning

Threats in Federated Learning. One of the most critical security threats in FL arises from **malicious or unreliable clients** uploading **poisoned or manipulated model updates**. Such poisoning and backdoor attacks can compromise the *integrity* of the global model, leading to biased predictions or even full model takeover.

FL aggregates model updates from a large number of distributed clients without directly verifying the correctness or reliability of their local data. This open-participation nature makes FL inherently vulnerable to *poisoning attacks*, where adversarial clients intentionally upload corrupted or biased gradients during the training stage [1], [19]. These malicious updates may affect the *local model*, the *shared dataset*, or ultimately corrupt the *aggregated global model*, resulting in degraded performance or targeted misclassification.

Data poisoning attacks typically involve manipulating local datasets or labels to mislead global optimization [20], [21]. In contrast, **model poisoning attacks** directly modify local parameters before uploading, causing the global model to deviate from its intended convergence path.

Backdoor attacks represent a stealthy variant of model poisoning. Without noticeably degrading the main-task accuracy, adversaries inject a *malicious subtask* into the model during training. This is often achieved by embedding specific patterns (known as *triggers*) into a small subset of poisoned samples, thereby *implanting a hidden backdoor*. The backdoor is later activated only when input data contain the same trigger pattern.

Backdoor poisoning attacks can be viewed as a hybrid threat that exploits data poisoning to implant a malicious function into the global model. Such attacks are particularly dangerous in cross-device FL scenarios, where detecting poisoned updates is challenging due to heterogeneous and asynchronous client participation.

A. Robust Aggregation and Anti-poisoning Defenses

FoolsGold [22] introduces a *robust aggregation mechanism* for federated learning (FL) that requires no prior assumption on the proportion of honest clients and relies solely on internal training dynamics to adaptively adjust each client's learning rate. The core idea is to assign *trust weights* to clients based on the cosine similarity of their historical gradient updates, thereby effectively mitigating *Sybil-based*, *label-flipping*, and *backdoor poisoning* attacks while preserving the contributions of honest participants.

1) *Motivations:* In federated learning, Sybil clients controlled by a common adversary tend to upload highly similar gradient updates, while honest clients exhibit diverse update directions due to heterogeneous and non-IID local data. Traditional robust aggregation schemes, such as Multi-Krum or Trimmed Mean, fail to defend against such coordinated attacks when the number of Sybils grows large.

2) *FoolsGold Algorithm Structure:* **FoolsGold** introduces a dynamic trust-based aggregation framework that assigns adaptive learning rates to clients based on the cosine similarity of their historical gradient updates. Clients with unique and uncorrelated update patterns retain higher learning rates, whereas those showing high similarity are down-weighted to mitigate the impact of Sybil-based poisoning. The detailed workflow of the algorithm can be summarized as follows:

- 1) **Historical update accumulation:** Each client i maintains a cumulative update vector

$$H_i = \sum_{t=1}^T \Delta_{i,t},$$

where $\Delta_{i,t}$ is the local model update of client i at iteration t . H_i records the historical contribution of client i across all iterations.

- 2) **Pairwise similarity computation:** The server computes the cosine similarity between clients i and j as

$$cs_{ij} = \cos(H_i, H_j) = \frac{H_i \cdot H_j}{\|H_i\| \|H_j\|},$$

where $cs_{ij} \in [-1, 1]$ measures the directional similarity of their cumulative updates.

- 3) **Maximum similarity extraction:** For each client i , the maximum similarity value is calculated as

$$v_i = \max_j(cs_{ij}),$$

where v_i denotes the strongest alignment of client i with any other client.

- 4) **Pardoning mechanism:** To balance similarity among clients, the cosine similarity is rescaled by

$$cs_{ij} \leftarrow cs_{ij} \times \frac{v_i}{v_j}, \quad \text{if } v_j > v_i,$$

where v_i and v_j are the maximum similarity scores of clients i and j .

- 5) **Trust weight derivation:** Each client’s preliminary trust weight is computed as

$$\alpha_i = 1 - \max_j(cs_{ij}),$$

and normalized by

$$\alpha_i = \frac{\alpha_i}{\max_i(\alpha_i)}.$$

- 6) **Logit scaling:** The trust weights are further adjusted using a logit function with confidence parameter κ :

$$\alpha_i = \kappa \left(\ln \frac{\alpha_i}{1 - \alpha_i} + 0.5 \right),$$

where α_i is clipped to $[0, 1]$ to maintain bounded values.

- 7) **Global model update:** The global parameters are updated using the weighted aggregation:

$$w_{t+1} = w_t + \sum_i \alpha_i \Delta_{i,t},$$

where w_t is the global model at iteration t .

3) **Optimization Formulation:** From an optimization perspective, the goal of FoolsGold is to obtain a global model update that maximizes the contribution of honest clients while suppressing the influence of colluding Sybils that generate highly similar gradient updates. Let $\Delta_{i,t}$ denote the local gradient update of client i at iteration t , and $\alpha_i \in [0, 1]$ denote its adaptive trust weight. The global optimization objective can be formulated as:

$$\min_{\{\alpha_i\}} \mathcal{L}(w_t + \sum_i \alpha_i \Delta_{i,t}) + \lambda \sum_{i,j} \alpha_i \alpha_j cs_{ij},$$

where $\mathcal{L}(\cdot)$ is the global loss function, cs_{ij} is the cosine similarity between clients i and j , and λ is a regularization coefficient controlling the penalty for correlated updates.

The first term enforces model convergence through federated aggregation, while the second term discourages the joint influence of clients whose historical updates are strongly aligned (i.e., likely Sybils). Thus, the optimization implicitly balances two competing objectives:

- **Minimize training loss:** drive global parameters toward optimal performance on the distributed data.
- **Minimize update similarity:** reduce the effective contribution of clients with high pairwise similarity.

The adaptive weighting rule in FoolsGold serves as an approximate solver for this problem, iteratively updating α_i based on cosine similarity and logit scaling:

$$\alpha_i^{(t+1)} = \kappa \left(\ln \frac{1 - \max_j(cs_{ij}^{(t)})}{\max_j(cs_{ij}^{(t)})} + 0.5 \right),$$

where κ controls the confidence level of trust adjustment. Substituting α_i into the global aggregation step,

$$w_{t+1} = w_t + \sum_i \alpha_i^{(t+1)} \Delta_{i,t},$$

the algorithm effectively performs an adaptive, similarity-regularized SGD update.

This formulation highlights that FoolsGold acts as a *trust-weighted optimization framework* that penalizes redundant or collusive updates while maintaining unbiased convergence toward the true global optimum.

4) **Results and Findings:** FoolsGold achieves over **99% attack mitigation** under Sybil-based, label-flipping, and backdoor scenarios, maintaining comparable accuracy to standard FL even with up to 9 colluding Sybils and across IID/non-IID data. The framework introduces only **1.5× CPU slowdown** and negligible GPU overhead.

B. Model Pruning for Efficient and Secure Federated Learning

Fine-Pruning [23] introduces an **adaptive and distributed parameter pruning mechanism** to improve both efficiency and security in federated learning (FL). By combining structured pruning with fine-tuning, it reduces model size, lowers communication cost, and removes dormant neurons that may encode backdoor triggers—thereby enhancing robustness against malicious updates while maintaining high accuracy.

1) **Motivations:** In federated learning (FL), clients often have limited computation and communication resources, making large-scale model training inefficient and vulnerable to *poisoning* or *backdoor* attacks. While traditional pruning focuses on reducing model size and cost, **Fine-Pruning** extends this concept by removing inactive neurons that are often linked to malicious triggers, thereby improving both efficiency and model integrity. Thus, pruning serves as a unified approach for **optimization** and **security enhancement** in FL.

2) **Pruning as a Defense Mechanism in Federated Learning:** Beyond improving model efficiency, pruning can also enhance security in federated learning (FL). By removing redundant or low-activation neurons, pruning reduces the network’s capacity to retain hidden backdoor triggers or poisoned patterns introduced by malicious clients. This weakens potential attack pathways while preserving model accuracy. Furthermore, smaller pruned models lower communication cost and limit the attack surface, making FL more robust and trustworthy.

3) **Fine-Pruning Algorithm Structure:** The Fine-Pruning process consists of two main stages: pruning and fine-tuning.

- 1) **Pruning Phase:** The defender executes the received model on a clean validation set D_{valid} , recording the average activation $a_i = \mathbb{E}_{x \in D_{\text{valid}}}[\phi_i(x)]$ of each neuron i . Neurons with the lowest activations are iteratively pruned in ascending order of a_i . The pruning stops once the validation accuracy drops below a predefined threshold (typically 4%), ensuring minimal impact on the clean-task performance.
- 2) **Fine-Tuning Phase:** The pruned model is fine-tuned using clean data with a small learning rate to restore accuracy and stability. This step also updates neurons that may have been partially involved in both clean and backdoor behaviors, further reducing the possibility of hidden triggers.

4) *Optimization Formulation*: The Fine-Pruning method can be viewed as a sparsity-constrained optimization problem that jointly minimizes classification loss, backdoor activation, and model complexity:

$$\min_{\mathbf{w}, \mathbf{s}} \mathcal{L}_{\text{cls}}(D_{\text{clean}}; \mathbf{w}, \mathbf{s}) + \lambda \mathcal{L}_{\text{bd}}(D_{\text{poison}}; \mathbf{w}, \mathbf{s}) + \beta \|\mathbf{s}\|_0,$$

where $\mathbf{s} \in \{0, 1\}^n$ represents neuron selection indicators, $\mathcal{L}_{\text{cls}}$ is the classification loss on clean data, $\mathcal{L}_{\text{bd}}$ measures backdoor activation, and λ, β are regularization coefficients balancing accuracy, robustness, and sparsity.

During pruning, neurons with lower average activation are set to zero ($s_i = 0$); fine-tuning then updates the retained neurons to refine performance on clean data. This process acts as an implicit form of optimization-aware regularization that aligns security and efficiency goals.

5) *Results and Findings*: Empirical evaluation on benchmark tasks such as face, speech, and traffic sign recognition demonstrates that Fine-Pruning achieves strong resistance against both standard and pruning-aware backdoor attacks. Specifically:

- For face recognition, the attack success rate decreased from 100% to 0% without affecting clean accuracy.
- For speech recognition, Fine-Pruning reduced the success rate from 77% to 2%, while maintaining over 98% accuracy on benign data.
- For traffic sign detection, the untargeted attack success dropped from 99% to 29%.

Compared with simple fine-tuning, which fails to update dormant backdoor neurons, Fine-Pruning successfully eliminates malicious activations while preserving model utility. Moreover, its computation cost is moderate—requiring minutes rather than days—making it practical for real-world FL systems with constrained resources.

C. Differential Privacy for Federated Learning

NbAFL (*Noising-before-Aggregation Federated Learning*) [24] introduces a differential privacy based framework to prevent information leakage in federated learning. It ensures privacy preservation by injecting calibrated Gaussian noise into local model updates *before* aggregation, thus protecting clients' private data while maintaining strong convergence guarantees. NbAFL achieves a formal (ϵ, δ) -DP guarantee, providing rigorous privacy control for both uplink (client-to-server) and downlink (server-to-client) communications in FL.

1) *Privacy Threats and Differential Privacy Defense*: In Federated Learning, although raw data stays local, shared model updates may still leak private information through model inversion or membership inference attacks. **Differential Privacy (DP)** addresses this by adding calibrated noise to model updates, masking individual contributions and ensuring that each client's participation has minimal impact on the aggregated model—thus providing provable protection against information leakage.

2) *Motivations*: While traditional FL avoids raw data sharing, it is still vulnerable to *information leakage from gradient exchanges*. Existing DP-based FL schemes often inject noise *after* aggregation at the server side, which only protects aggregated updates but not individual client uploads. **NbAFL** is motivated by the need to:

- Strengthen privacy guarantees by adding noise locally at each client before model aggregation;
- Achieve a global (ϵ, δ) -DP guarantee covering both uplink and downlink communications;
- Maintain stable convergence and learning accuracy despite noise perturbations.

3) *NbAFL Algorithm Structure*: The overall NbAFL workflow is summarized as follows:

- 1) **Initialization**: The server initializes the global model $w^{(0)}$ and broadcasts it to all clients, together with the desired privacy parameters (ϵ, δ) .
- 2) **Local training**: Each client i trains the model on its private data D_i and obtains local parameters $w_i^{(t)}$ by minimizing its empirical loss function:

$$w_i^{(t)} = \arg \min_{w_i} (F_i(w_i) + \frac{\mu}{2} \|w_i - w^{(t-1)}\|^2),$$

where μ controls the proximal regularization.

- 3) **Gradient clipping**: The client clips parameter norms to a threshold C :

$$w_i^{(t)} \leftarrow \frac{w_i^{(t)}}{\max(1, \|w_i^{(t)}\|/C)}.$$

- 4) **Noise injection**: Each client perturbs the clipped parameters by adding Gaussian noise:

$$\tilde{w}_i^{(t)} = w_i^{(t)} + n_i^{(t)}, \quad n_i^{(t)} \sim \mathcal{N}(0, \sigma_U^2 I),$$

ensuring client-side (ϵ, δ) -DP.

- 5) **Secure aggregation**: The server aggregates the noised parameters from all N clients:

$$w^{(t)} = \sum_{i=1}^N p_i \tilde{w}_i^{(t)},$$

and optionally adds server-side noise $n_D^{(t)} \sim \mathcal{N}(0, \sigma_D^2 I)$ before broadcasting.

- 6) **Broadcasting and updating**: The perturbed global model is sent back to all clients for the next round.

4) *Optimization Formulation*: From an optimization perspective, NbAFL aims to minimize the global empirical risk while satisfying a privacy constraint:

$$\min_w \sum_{i=1}^N p_i F_i(w, D_i) \quad \text{s.t.} \quad M(D_i) \text{ satisfies } (\epsilon, \delta)\text{-DP}.$$

Here, $M(\cdot)$ represents the randomized mechanism of noising each client's model update. The noise variance is chosen based on the Gaussian mechanism:

$$\sigma_U \geq \frac{c \Delta s_U}{\epsilon}, \quad \text{where} \quad \Delta s_U = \frac{2C}{|D_i|}.$$

The analysis proves that adding Gaussian noise under this condition guarantees global (ϵ, δ) -DP while bounding the convergence loss.

Furthermore, the authors derive a theoretical convergence upper bound:

$$\mathbb{E}[F(w_T)] - F(w^*) \leq \mathcal{O}\left(\frac{1}{T}\right) + \mathcal{O}(\sigma_U^2),$$

revealing that larger noise (stronger privacy) degrades convergence, highlighting the tradeoff between privacy and utility.

5) *Results and Findings::* Extensive experiments on the MNIST dataset validate the theoretical findings:

- **Privacy–Convergence Tradeoff:** Relaxing privacy constraints ($\epsilon \uparrow$) improves convergence and lowers training loss.
- **Client and Round Effects:** Increasing the number of clients N and aggregation rounds T improves stability and accuracy under a fixed privacy level.
- **Optimal Clipping and Scheduling:** There exists an optimal clipping threshold C^* and an optimal number of participating clients K^* that jointly minimize loss while preserving privacy.

V. COMPREHENSIVE ANALYSIS OF RECENT ADVANCEMENTS

This section illustrates a high level comparison among the reviewed works and studies the trade-offs between the required QoS and system complexity.

A. Discussion on Latency Minimization Methods

The reviewed latency-minimization methods target the same bottleneck, but each of them focuses on a different part of the system. FedCS reduces the round duration by selecting the clients that can finish both computation and upload within the round deadline. This makes the protocol efficient and lightweight for practical settings and resolves the straggler problem. However, weaker clients may never participate, affecting fairness across users.

The multihop optimization framework tackles latency from a more detailed system-level perspective. By jointly optimizing routing decisions, CPU frequencies, and transmit power, it reduces the end-to-end delay in hierarchical wireless networks. Compared to FedCS, this approach achieves larger latency improvements but comes with higher computational complexity. It relies on iterative optimization and requires a central controller, making it less efficient for highly dynamic environments due to the high complexity of optimization.

WHALE-FL provides a more adaptive solution by adjusting the model size based on device conditions. This method reduces latency without excluding devices, since each client trains a subnetwork aligned with its capabilities. It works well in heterogeneous and mobile scenarios and shows significant improvement in convergence time. The drawback is that shrinking subnetworks introduces additional communication overhead to inform the users which subnetwork they should utilize in each round.

Overall, FedCS offers a simple and lightweight improvement, the multihop optimization achieves deeper latency reductions at the cost of complexity, and WHALE-FL provides a flexible and adaptive strategy for mobile environments.

B. Discussion on Communication-Efficient FL

The communication-efficient frameworks reduce the overhead of transmitting gradients or model parameters with distinct approaches. FedPAQ uses periodic averaging and quantization to reduce the number of communication rounds. AGQ improves quantization by adapting the bit-width of each client based on its link quality and gradient distribution, making it more stable under heterogeneous conditions. DAdaQuant further extends this idea by adapting quantization over time and across clients, and its error-feedback mechanism keeps the accuracy stable even under very high compression.

FedAT and CHFL reduce communication by changing how the system is structured. FedAT uses asynchronous multi-tier aggregation, which removes synchronization delays and allows faster clients to contribute more often. CHFL applies hierarchical compression, reducing the amount of data transmitted across long-range cloud links.

In short, quantization-based methods are easy to integrate and work well with most models, while asynchronous and hierarchical methods provide larger scalability benefits but require more coordination.

C. Discussion on Secure FL

The secure FL methods defend against different types of attacks. FoolsGold targets poisoning attacks by detecting clients whose gradients look too similar. It reduces the impact of Sybil and backdoor attacks using adaptive trust weights and does not require any assumptions about the number of adversaries. This approach is lightweight and easy to use but may not detect highly optimized or stealthy attacks.

Fine-Pruning improves security by removing inactive neurons that can store backdoor triggers. It also reduces model size, which helps communication and computation. The limitation is that more advanced backdoor strategies may not always rely on dormant neurons.

NbAFL protects privacy using differential privacy. It adds noise locally at each client before aggregation, guaranteeing that private data cannot be recovered from gradients. The trade-off is that stronger privacy means more noise, which slows convergence.

Overall, FoolsGold is the most effective for poisoning resistance, Fine-Pruning is the most useful for backdoor removal with the added benefit of compression, and NbAFL is the best choice when formal privacy guarantees are required.

D. Cross-Directional Comparison

Across the three directions, we observe that some methods are naturally against each other. For instance, quantization-based methods from communication-efficient FL can be combined with FedCS or WHALE-FL to jointly improve latency

TABLE I
COMPARISON OF REVIEWED WORKS ACROSS LATENCY, COMMUNICATION EFFICIENCY, AND SECURITY ISSUES

Work	Latency	Comm. Eff.	Security	Framework Summary	Contribution
FedCS [9]	✓	✗	✗	Client selection under round deadlines	Time-to-accuracy reduced by ~75 min
PAFL Multihop [11]	✓	✗	✗	Joint routing/CPU/power optimization in multihop FL via alternation minimization	Latency reduced by 69%, Energy harvesting
WHALE-FL [13]	✓	✗	✗	Adaptive subnetwork selection per device condition	1.9× faster Convergence
FedPAQ [14]	✓	✓	✗	Periodic averaging + quantized updates	40× less communication, lower convergence time
AGQ [15]	✗	✓	✗	Adaptive quantization using SNR + variance	60% bandwidth savings
DAdaQuant [16]	✓	✓	✗	Time and client adaptive quantization, error feedback	Up to 95% comm. reduction, faster training
FedAT [17]	✓	✓	✗	Asynchronous multi-tier aggregation (no waiting for stragglers)	45% less comm., 3.2× faster convergence
CHFL [18]	✓	✓	✗	Hierarchical client-edge-cloud compression	80% less bandwidth consumption, faster convergence
FoolsGold [22]	✗	✗	✓	Similarity based trust weighting for poisoning defense	99% attack mitigation
Fine-Pruning [23]	✗	✗	✓	Pruning inactive neurons + fine-tuning	Backdoor success reduced to 0%
NbAFL [24]	✗	✗	✓	Local DP noise injection before aggregation	Formal DP guarantee with controlled accuracy loss

and bandwidth usage. However, security methods often introduce additional overhead, such as the additional noise for differential privacy methodology.

Frameworks proposed for resolving latency and communication concerns mostly focus on improving the efficiency of FL in terms of latency and communication overhead under certain performance constraints, while methodologies proposed for secure FL focus on robustness against adversaries or privacy leakage. For real NextG deployments, a combination of these methods is needed. A typical system may use a lightweight client-selection strategy for latency, adaptive quantization for communication overhead reduction, and a simple trust-based defense like FoolsGold for security. Table I briefly illustrates the contribution and the main idea behind different methodologies reviewed in this work.

VI. CONCLUSION AND FUTURE WORKS

In this survey paper, we showed the increasing demand for incorporating FL in NextG wireless networks, demonstrating that future wireless networks are required to support ubiquitous intelligence and massive device connectivity. To meet the QoS required for such applications, we introduced three major research directions that play a pivotal role for model optimization in FL over NextG wireless networks, i.e., latency minimization, communication efficiency, and secure FL methodologies. After reviewing a few of the recent works addressing these demands, we provided a comprehensive comparison among them and elaborated on their contributions. This work explicitly emphasizes on the importance of reliable, ultra-low latency, and secure FL methodologies for 6G and beyond, envisioned to support a large spectrum of intelligent applications. Although FL system deployment requires fast, reliable and secure frameworks, the intersection between these

directions is largely overlooked and requires further exploration.

REFERENCES

- [1] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, “Advances and open problems in federated learning,” *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.
- [3] Z. Yang, M. Chen, K.-K. Wong, H. V. Poor, and S. Cui, “Federated learning for 6g: Applications, challenges, and opportunities,” *Engineering*, vol. 8, pp. 33–41, 2022.
- [4] M. Bennis, M. Debbah, and H. V. Poor, “Ultrareliable and low-latency wireless communication: Tail, risk, and scale,” *Proceedings of the IEEE*, vol. 106, no. 10, pp. 1834–1853, 2018.
- [5] Q. Yang, Y. Liu, T. Chen, and Y. Tong, “Federated machine learning: Concept and applications,” *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.
- [6] R. Li, H. Xu, G. Zhang, Y. Chen, and X. Chen, “Latency optimization for federated learning over wireless power transfer,” *Physical Communication*, vol. 53, p. 101765, 2022.
- [7] J. Du, B. Jiang, C. Jiang, Y. Shi, and Z. Han, “Gradient and channel aware dynamic scheduling for over-the-air computation in federated edge learning systems,” *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 4, pp. 1035–1050, 2023.
- [8] Z. Wang, Q. Hu, M. Xu, Y. Zhuang, Y. Wang, and X. Cheng, “A systematic survey of blockchained federated learning,” *arXiv preprint arXiv:2110.02182*, 2021.
- [9] T. Nishio and R. Yonetani, “Client selection for federated learning with heterogeneous resources in mobile edge,” in *ICC 2019-2019 IEEE international conference on communications (ICC)*. IEEE, 2019, pp. 1–7.
- [10] M. Sviridenko, “A note on maximizing a submodular set function subject to a knapsack constraint,” *Operations Research Letters*, vol. 32, no. 1, pp. 41–43, 2004.
- [11] S. Shaon, V.-D. Nguyen, and D. C. Nguyen, “Latency optimization for wireless federated learning in multihop networks,” *IEEE Transactions on Vehicular Technology*, 2025.
- [12] U. Niesen, D. Shah, and G. Wornell, “Adaptive alternating minimization algorithms,” in *2007 IEEE International Symposium on Information Theory*. IEEE, 2007, pp. 1641–1645.
- [13] H.-a. Su, J. Geng, L. Li, X. Qin, Y. Hou, H. Wang, X. Fu, and M. Pan, “Whale-fl: Wireless and heterogeneity aware latency efficient federated learning over mobile devices via adaptive subnetwork scheduling,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 19, 2025, pp. 20 619–20 627.
- [14] A. Reiszadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, “Fedpaq: A communication-efficient federated learning method with periodic averaging and quantization,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 10, pp. 4226–4239, 2020.
- [15] X. Zhang, M. Xu, C. Wang, Z. Wang, and T. Chen, “Communication-efficient federated learning for heterogeneous edge devices based on adaptive gradient quantization,” *IEEE Internet of Things Journal*, vol. 11, no. 12, pp. 11 234–11 248, 2024.
- [16] R. Hönig, Y. Zhao, and R. Mullins, “Dadaquant: Doubly-adaptive quantization for communication-efficient federated learning,” *arXiv preprint arXiv:2111.00465*, 2022, presented at ICLR 2022.
- [17] Z. Chai, Y. Chen, A. Anwar, L. Zhao, Y. Cheng, and H. Rangwala, “Fedat: A high-performance and communication-efficient federated learning system with asynchronous tiers,” in *SC21: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–17.
- [18] Y. Liu, Z. Qu, and J. Wang, “Compressed hierarchical federated learning for edge-level imbalanced wireless networks,” *IEEE Transactions on Computational Social Systems*, vol. 12, no. 5, pp. 3131–3142, 2025.
- [19] Z. Tian, L. Cui, J. Liang, and S. Yu, “A comprehensive survey on poisoning attacks and countermeasures in machine learning,” *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–35, 2022.
- [20] J. Feng, Q.-Z. Cai, and Z.-H. Zhou, “Learning to confuse: Generating training time adversarial data with auto-encoder,” *arXiv preprint arXiv:1905.09027*, 2019.
- [21] L. Muñoz-González, B. Biggio, A. Demontis, A. Paudice, V. Wongrasamee, E. C. Lupu, and F. Roli, “Towards poisoning of deep learning algorithms with back-gradient optimization,” in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISec ’17)*. ACM, 2017, pp. 27–38.
- [22] C. Fung, C. J. Yoon, and I. Beschastnikh, “Mitigating sybils in federated learning poisoning,” *arXiv preprint arXiv:1808.04866*, 2018.
- [23] K. Liu, B. Dolan-Gavitt, and S. Garg, “Fine-pruning: Defending against backdooring attacks on deep neural networks,” in *International symposium on research in attacks, intrusions, and defenses*. Springer, 2018, pp. 273–294.
- [24] K. Wei, J. Li, M. Ding, C. Ma, H. H. Yang, F. Farokhi, S. Jin, T. Q. Quek, and H. V. Poor, “Federated learning with differential privacy: Algorithms and performance analysis,” *IEEE transactions on information forensics and security*, vol. 15, pp. 3454–3469, 2020.